

download_data.py

Gold Price Data Downloader — Full Code Walkthrough

■ What This File Does

This Python script downloads 9 years of historical **Gold Futures (GC=F)** daily price data from Yahoo Finance and stores it as a CSV file. On every subsequent run it skips the download and reads directly from disk, making repeated use fast and offline-capable.

■ Full Source Code

```
import yfinance as yf
import pandas as pd
import os

DATA_FILE = "gold_price_data.csv"

def download_data():
    if os.path.exists(DATA_FILE):
        print(f"{DATA_FILE} already exists. Skipping download.")
        return pd.read_csv(DATA_FILE, index_col='Date', parse_dates=True)

    print("Downloading historical gold prices...")
    ticker = "GC=F"
    gold_data = yf.download(ticker, start="2015-01-01", end="2023-12-31")

    if gold_data.empty:
        print("Failed to download data.")
        return None

    if isinstance(gold_data.columns, pd.MultiIndex):
        gold_data.columns = gold_data.columns.droplevel('Ticker')

    gold_data.to_csv(DATA_FILE)
    print(f"Data saved to {DATA_FILE}")
    return gold_data

if __name__ == "__main__":
    df = download_data()
    if df is not None:
        print(df.head())
```

■ Line-by-Line Code Explanation

Imports

`import yfinance as yf`

Loads the yfinance library. This library acts as a wrapper around Yahoo Finance's API, letting us download historical market data (prices, volume, etc.) for any ticker symbol with a single function call.

`import pandas as pd`

Pandas is the core data manipulation library. All data from yfinance comes as a pandas DataFrame — a spreadsheet-like structure with rows (dates) and columns (Open, High, Low, Close, Volume).

`import os`

Python's built-in OS module. We use `os.path.exists()` to check whether the CSV has already been downloaded so we can skip a redundant network request.

`DATA_FILE = "gold_price_data.csv"`

A module-level constant holding the filename used for both saving and loading the data. Defining it as a constant at the top makes it easy to change in one place without hunting through the code.

Inside `download_data()`

`if os.path.exists(DATA_FILE):`

Before touching the network, checks if the file is already on disk. If it exists, the function reads it directly with `pd.read_csv()` and returns immediately. `index_col='Date'` sets the Date column as the row index; `parse_dates=True` converts the date strings into real Python datetime objects for easy time-series operations later.

`ticker = "GC=F"`

GC=F is Yahoo Finance's symbol for Gold Futures — the most liquid gold price benchmark. Futures prices closely track physical gold spot prices.

`gold_data = yf.download(ticker, start="2015-01-01", end="2023-12-31")`

Calls Yahoo Finance's API to fetch daily OHLCV (Open, High, Low, Close, Volume) data for the 9-year window 2015–2023. The result is a pandas DataFrame with Date as the index and price columns. This is a blocking network call — it waits until the download completes.

`if gold_data.empty:`

After downloading, immediately validates that we actually received data. An empty DataFrame can happen if Yahoo Finance is unreachable, the ticker is wrong, or the date range returns no trading days. The function prints an error and returns None instead of crashing.

`if isinstance(gold_data.columns, pd.MultiIndex):`

Newer versions ($\geq 0.2.x$) of yfinance return a MultiIndex column like (Price, Ticker) — e.g. ('Close', 'GC=F'). This would break downstream code that expects simple column names like 'Close'. `droplevel('Ticker')` flattens it back to single-level column names.

`gold_data.to_csv(DATA_FILE)`

Writes the clean DataFrame to disk as a CSV. The Date index is automatically included as the first column. This file is what future runs load from disk, skipping the network entirely.

if __name__ == "__main__":

This block only runs when the script is executed directly (python download_data.py). When this file is imported as a module by another script, this block is skipped. It calls download_data() and prints the first 5 rows as a sanity check.

■ Data Flow

1

Script starts

Checks if gold_price_data.csv already exists on disk.

2

File exists → Load from disk

pd.read_csv() loads the CSV. Date column becomes the index, strings are parsed into datetime. Returns the DataFrame immediately — no internet needed.

3

File missing → Download

yf.download('GC=F', ...) makes an HTTP request to Yahoo Finance and downloads 9 years of daily OHLCV data.

4

Validate

Checks if the returned DataFrame is empty. If empty, prints error and returns None.

5

Fix columns

If columns are MultiIndex (newer yfinance), flattens them to simple names: Open, High, Low, Close, Volume, Adj Close.

6

Save & Return

Saves the clean DataFrame to gold_price_data.csv and returns it to the caller.

■ Output — What the CSV Looks Like

The saved CSV has the following structure:

Date	Open	High	Low	Close	Volume
2015-01-02	1206.80	1209.60	1199.00	1202.30	182400
2015-01-05	1200.10	1210.50	1196.40	1207.60	201500

...	...	...	...	...	...
-----	-----	-----	-----	-----	-----

■ Each row = one trading day. Non-trading days (weekends/holidays) are automatically excluded by Yahoo Finance.